

Правительство Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО” (Университет ИТМО)

Факультет Программной инженерии и компьютерной техники

ОТЧЕТ
ПО КУРСОВОЙ РАБОТЕ:
“Интерактивная доска itmo.board”

Проект выполняли:

Маликов Глеб Игоревич
Кобик Никита Алексеевич
Касьяненко Вера Михайловна
Михеева Яна Александровна
Кремпольская Екатерина Александровна

Санкт-Петербург
2024

Содержание

1. Цель работы
2. Техническое задание на программную систему
3. Анализ требований к программной системе
4. Описание структур данных, используемых в программной системе.
5. Схема Базы данных
6. Ссылка на figma
7. Интерфейсы
8. Список и описание алгоритмов с помощью которых планируется реализовать программную систему (в виде диаграмм деятельности)
9. Список и описание библиотек для реализации программной
10. Описание тестирования и результатов тестирования программной системы
11. Выводы

Цель работы:

Целью работы является создание веб-приложения рисовалки с коллаборативным редактированием.

Техническое задание на программную систему:

Мы предоставляем веб-приложение рисовалки с коллаборативным редактированием. Оно позволяет пользователям создавать, организовывать и визуализировать идеи на единой рабочей доске в режиме реального времени. Данный проект хранится на github, используя ссылку: [CandyGoose/itmo.board \(github.com\)](https://github.com/CandyGoose/itmo.board) или с помощью QR-code можно просмотреть данный проект



Рисунок 1. QR-code

Для достижения поставленной цели необходимо было решить следующие задачи:

- сформулировать основную идею приложения;
- выбрать стек технологий, на которых будет работать приложение;
- спроектировать основные страницы приложения;
- разработать интерфейсы приложения;
- написать серверную часть;
- провести тестирование;
- запустить приложение и проверить его работоспособность.

Анализ требований к программной системе:

Веб-приложение соблюдает принципы RESTful архитектуры:

- Проект состоит из двух частей: клиента и сервера. Они не имеют логической связанности между компонентами, следовательно они беспрепятственно могут быть заменены. Разработка этих частей велась отдельно.
- Отсутствие состояния клиента на сервере: сервер не хранит данные об авторизованных пользователях. Также отсутствуют промежуточные состояния при выполнении действий пользователями.
- Запросы разбиты по методам GET, POST, PUT, DELETE в зависимости от их назначения.
- Единообразие интерфейса: все запросы и ответы отправляются в JSON формате. Он используется для представления данных, их передачи, манипуляции и изменения. На сервере JSON предшествуют dict словари, на клиенте - JavaScript Object.

Основные функции:

Пользователь имеет возможность:

1. Выполнять регистрацию;
2. Входить в систему;
3. Выходить из системы;
4. Создавать организацию;
5. Удалять организацию;
6. Приглашать других пользователей в организацию;
7. Создавать доску;
8. Удалять доску;
9. Переименовывать доску;
10. Скопировать ссылку на доску;
11. Рисовать на доске;
12. Добавлять на доску фигуры;
13. Добавлять на доску текст;
14. Добавлять на доску изображения;
15. Удалять объекты с доски;
16. Перемещать объекты по слоям;
17. Перемещаться по истории изменений;
18. Изменять стиль объектов;
19. Экспортировать содержимое доски в формате PNG и SVG;
20. Переключаться между режимом просмотра и редактирования доски;
21. Поменять язык;
22. Поменять тему.

Описание структур данных, используемый в программной системе.

В клиентской части приложения данные хранятся с использованием Liveblocks Storage и zustand для управления данными канвы и инструментов.

1. Данные о пользователе:

- Токен доступа;
- Информация о пользователе (идентификатор, имя и права доступа).

2. Данные о состоянии доски:

- Камера (camera):
 - Позиция камеры (x, y);
 - Масштаб (scale).
- Состояние канвы (canvasState):
 - Описывает текущий режим работы:
 - None: бездействие;
 - SelectionNet: выбор нескольких слоев;
 - Translating: перемещение слоев;
 - Inserting: добавление нового слоя;
 - Resizing: изменение размеров слоя;
 - Pencil: рисование;
 - Pressing: перемещение камеры.
 - Дополнительные данные состояния:
 - current: текущая точка;
 - origin: начальная точка;
 - layerType: тип вставляемого слоя;
 - initialBounds и corner: изменение размеров.
- Слои хранятся в виде объектов Layer:
 - Общие поля:
 - id: уникальный идентификатор слоя;
 - type: тип слоя (например, Rectangle, Text, Image);
 - x, y: координаты на канве;
 - width, height: размеры слоя;
 - fill: цвет слоя (или null для прозрачности);
 - lineWidth: толщина линий (для фигур);
 - value: содержимое слоя (например, текст).
 - Специфические поля:
 - Для текста (TextLayer) и заметок (NoteLayer): шрифт, размер шрифта, выравнивание текста, форматирование;
 - Для изображения (ImageLayer): ссылка на изображение (src);
 - Для пути (PathLayer): массив точек (points).
 - Типы слоев:
 - RectangleLayer, EllipseLayer, PathLayer, TextLayer, NoteLayer, ImageLayer.
 - Порядок слоев (layerIds)
- Выделенные слои (selection):

- Массив идентификаторов слоев, которые выбраны пользователем.
- Черновик карандаша (pencilDraft):
 - Массив точек, формирующих текущую линию карандаша.

3. Данные об инструментах:

- Настройки инструментов:
 - Цвет (lastUsedColor): RGB-цвет для новых объектов.
 - Толщина линии (lineWidth): используется для рисования и фигур.
 - Шрифт (fontName) и размер шрифта (fontSize): для текстовых и заметочных слоев.
 - Выравнивание текста (textAlign): слева, по центру или справа.
 - Форматирование текста (textFormat): жирный, курсив, зачеркнутый.

На серверной стороне данные обрабатываются в виде объектов JavaScript, созданных на основе схем Mongoose. Для группировки данных в контроллерах (controllers) используются массивы объектов JavaScript. Перед отправкой клиенту объекты MongoDB преобразуются в обычные JavaScript-объекты, а затем сериализуются в JSON для унифицированной передачи данных.

Схема Базы данных

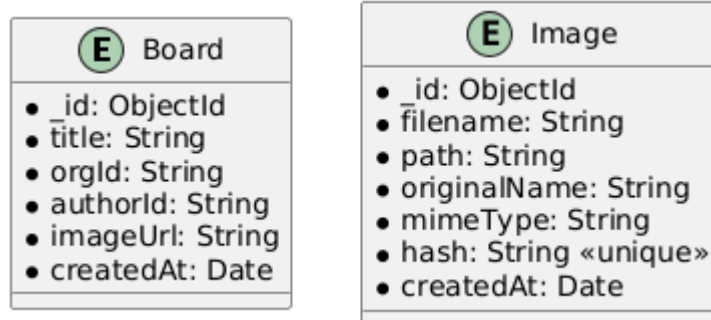


Рисунок 2. Даталогическая модель базы данных

Проектирование интерфейсов

Макет приложения -

<https://www.figma.com/design/b94Y02cAVRK6mj8qb5yAzB/itmo.board?node-id=415-1144&p=f&t=S0kWEYGGh8QquK9R-0>

Главный интерфейс веб-приложения



Рисунок 3. Основная страница

Регистрация в системе сайта

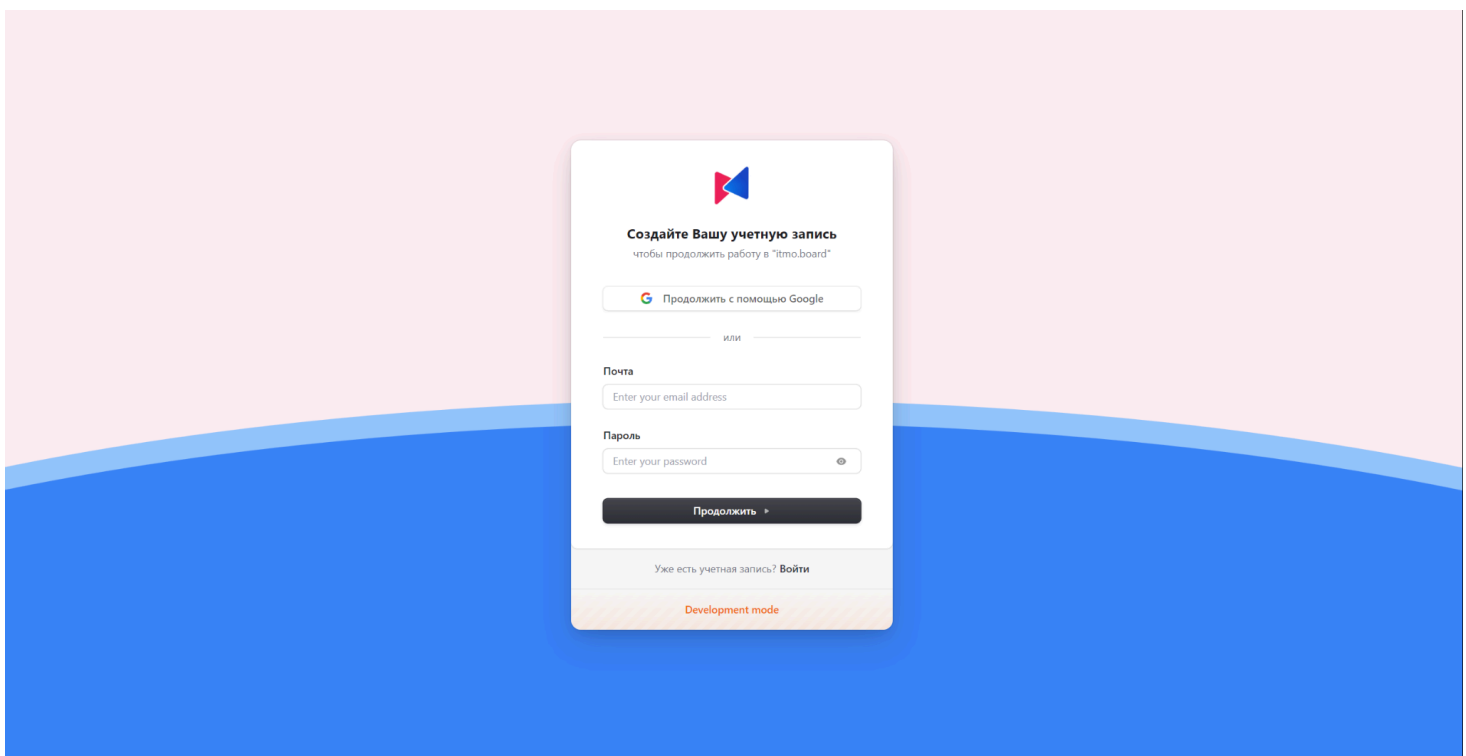


Рисунок 4. Регистрация в системе

Авторизация в системе

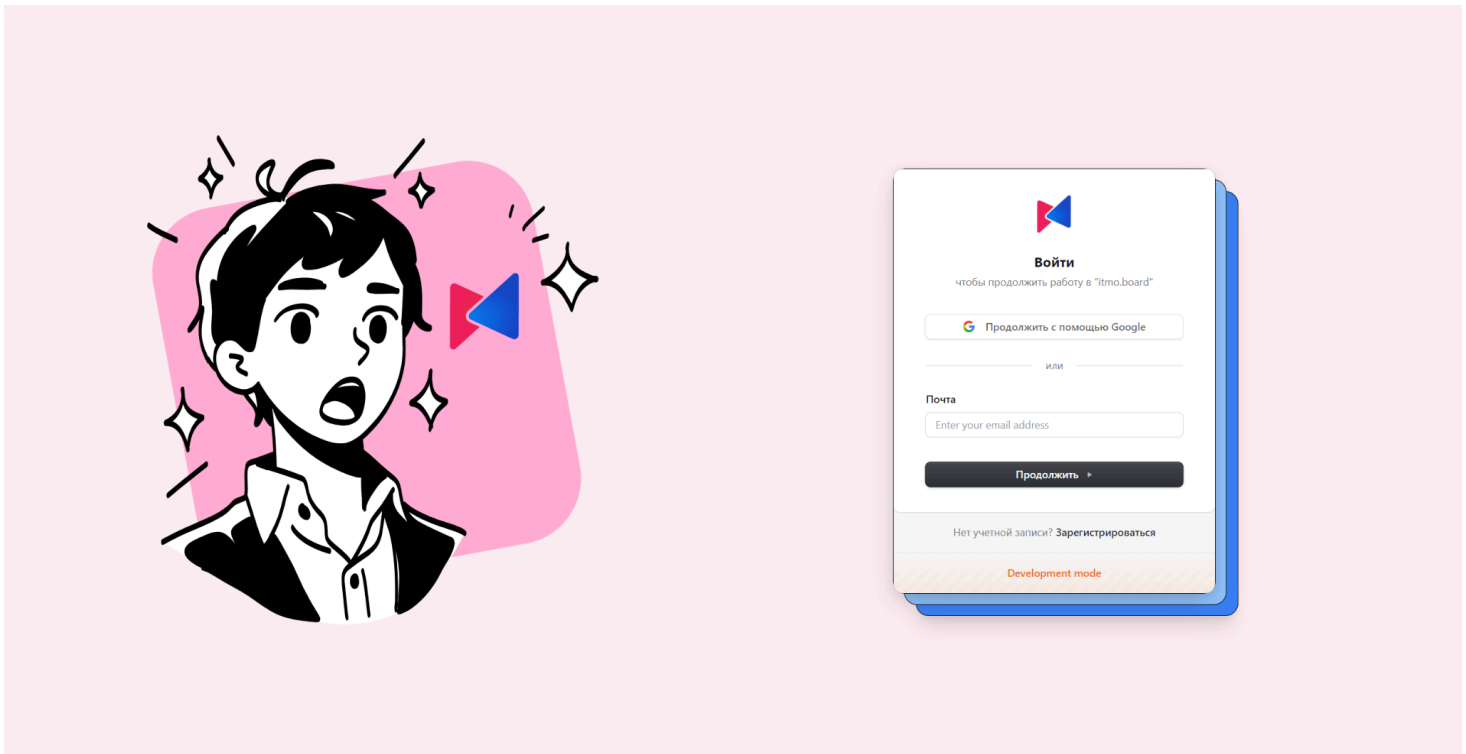


Рисунок 5. Авторизация в системе

Интерфейс выбора организации



Рисунок 6. Интерфейс выбора организации

Интерфейс доски

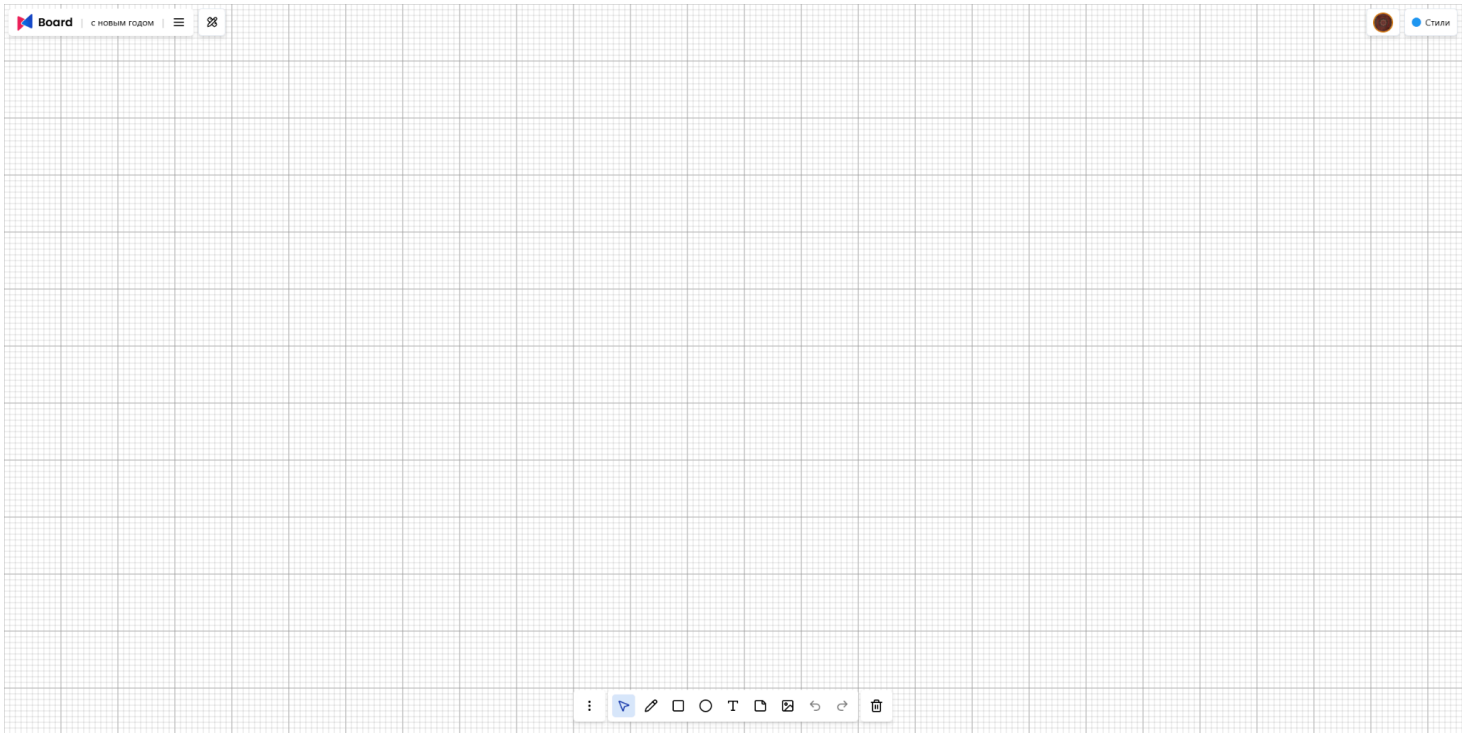


Рисунок 7. Интерфейс доски

Интерфейс настроек пользователя

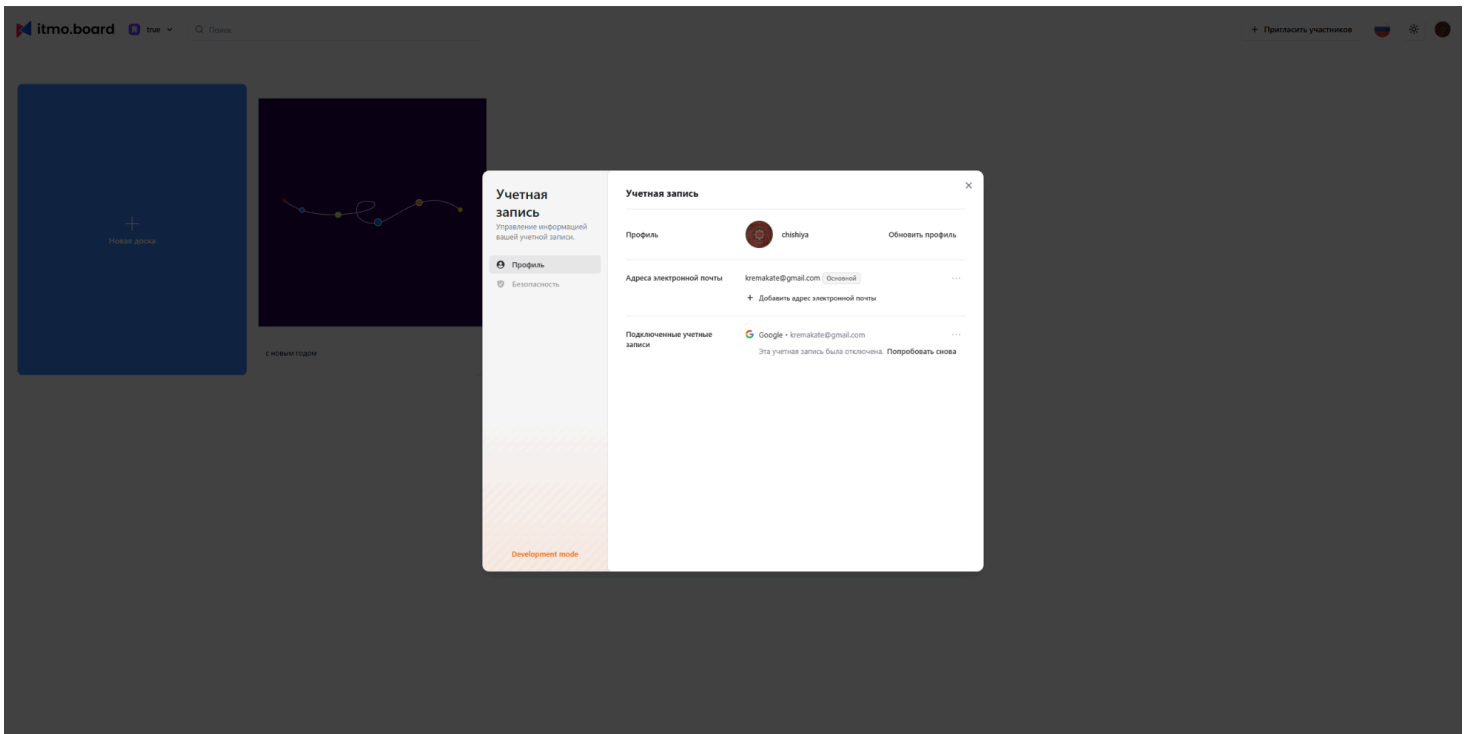


Рисунок 8. Интерфейс настроек пользователя

Интерфейс настроек организации

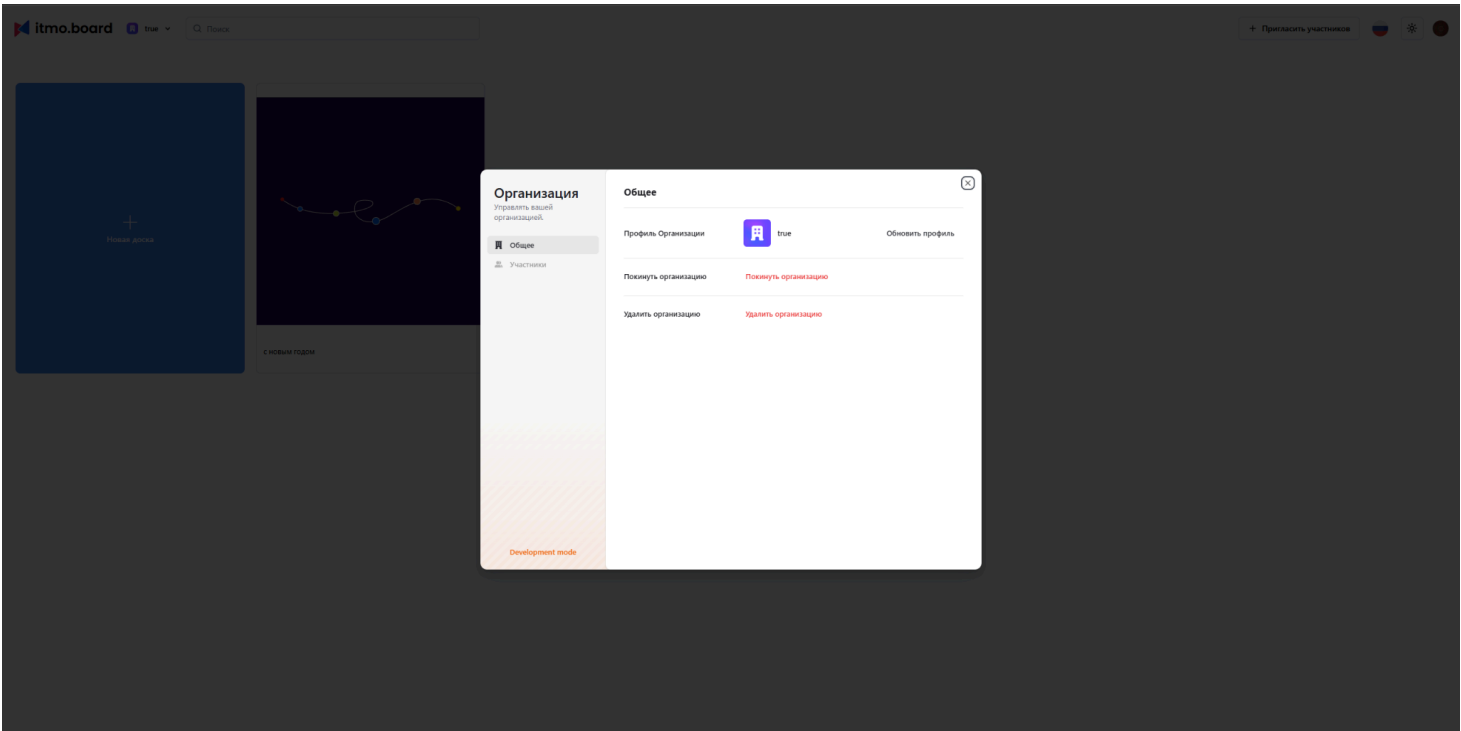


Рисунок 9. Интерфейс настроек организации

Интерфейс расширенных возможностей доски

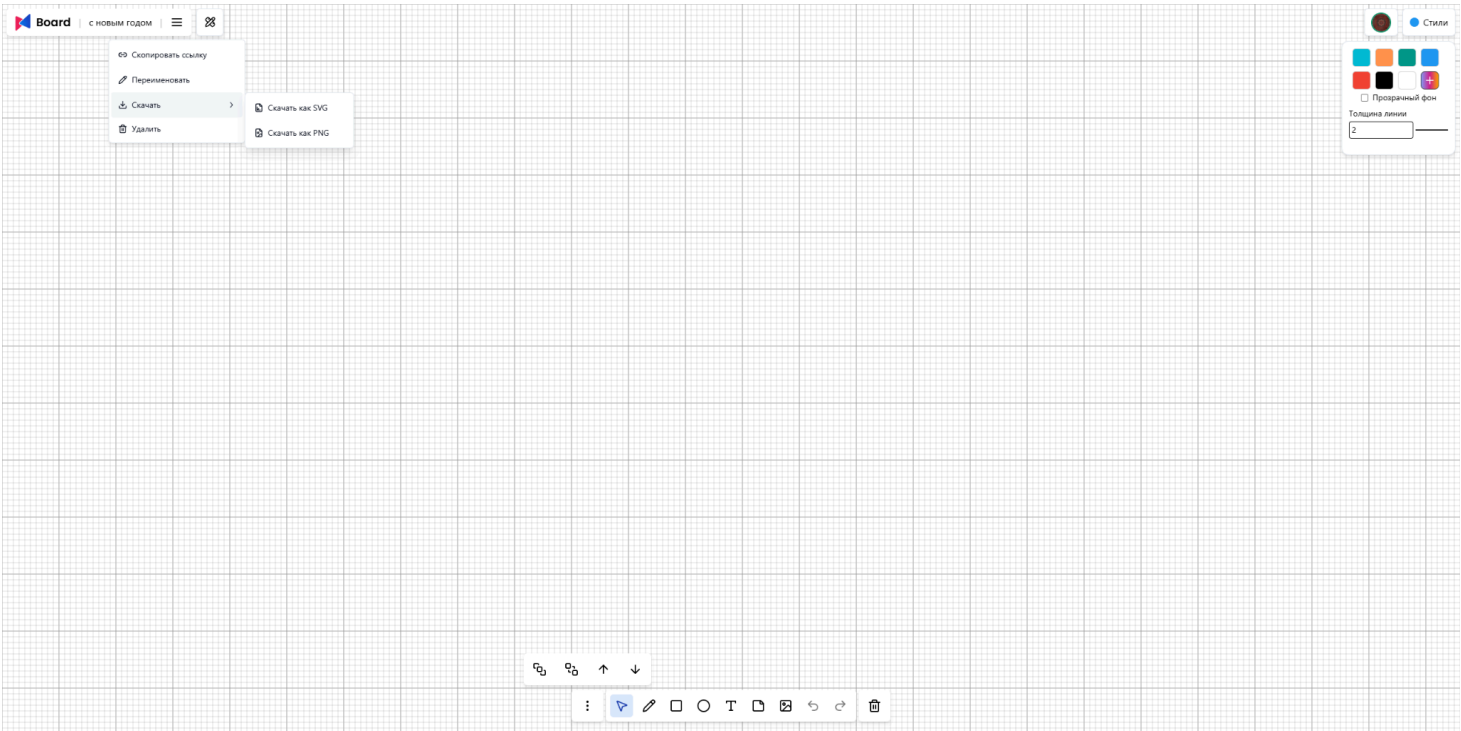


Рисунок 10. Интерфейс расширенных возможностей доски

Список и описание алгоритмов реализации

- Алгоритм входа в аккаунт

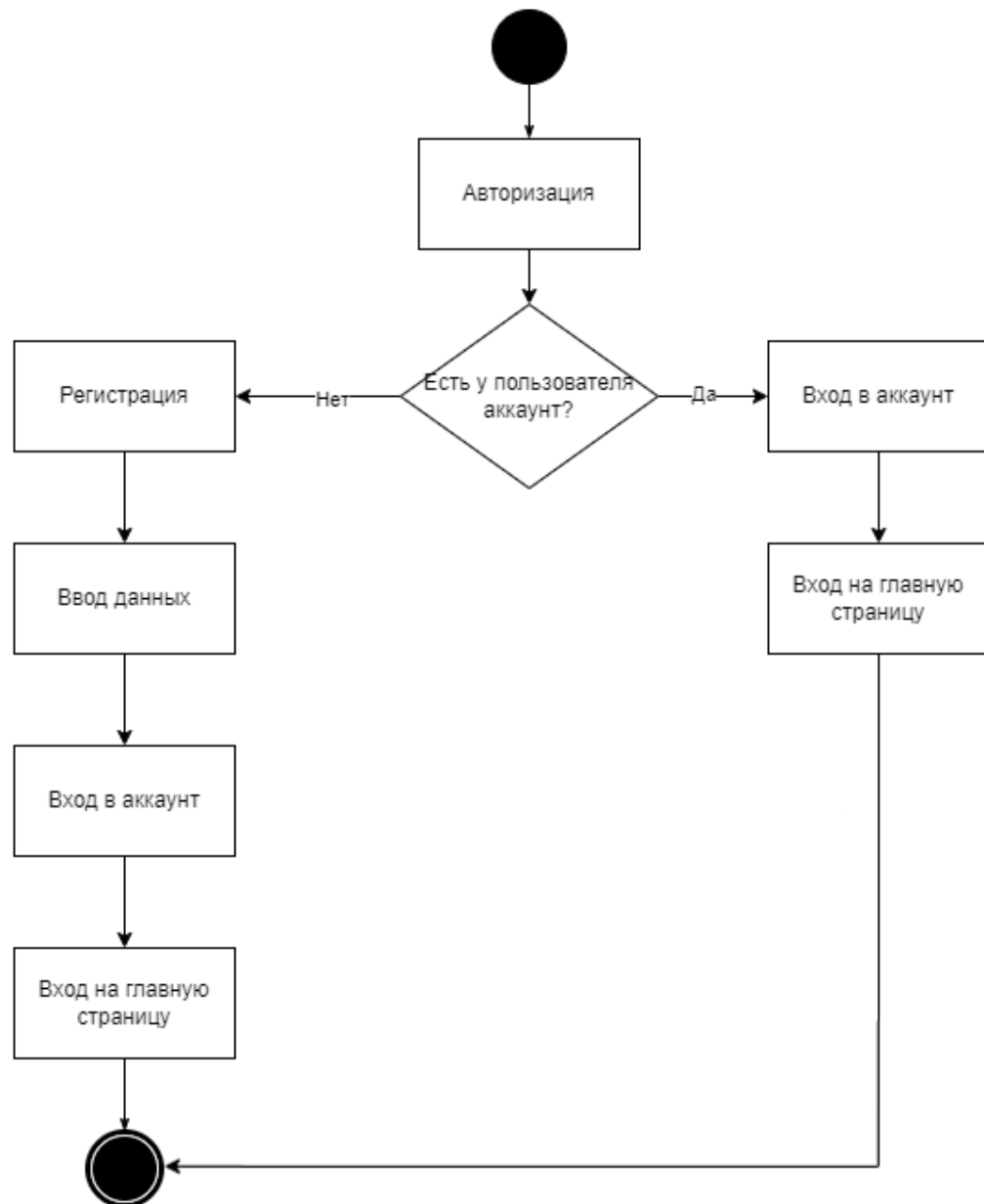


Рисунок 11. Диаграмма деятельности для алгоритма

- Алгоритм действий на главной странице

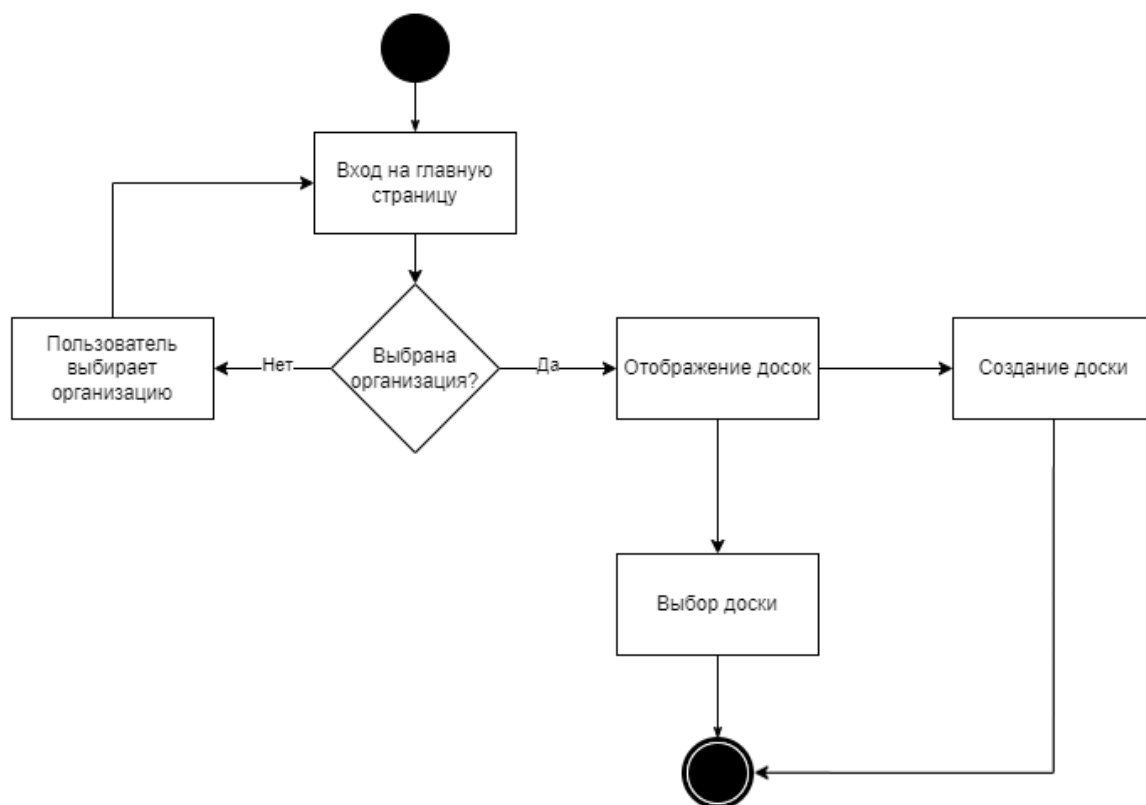


Рисунок 12. Диаграмма деятельности для алгоритма

- **Алгоритм действий на доске**

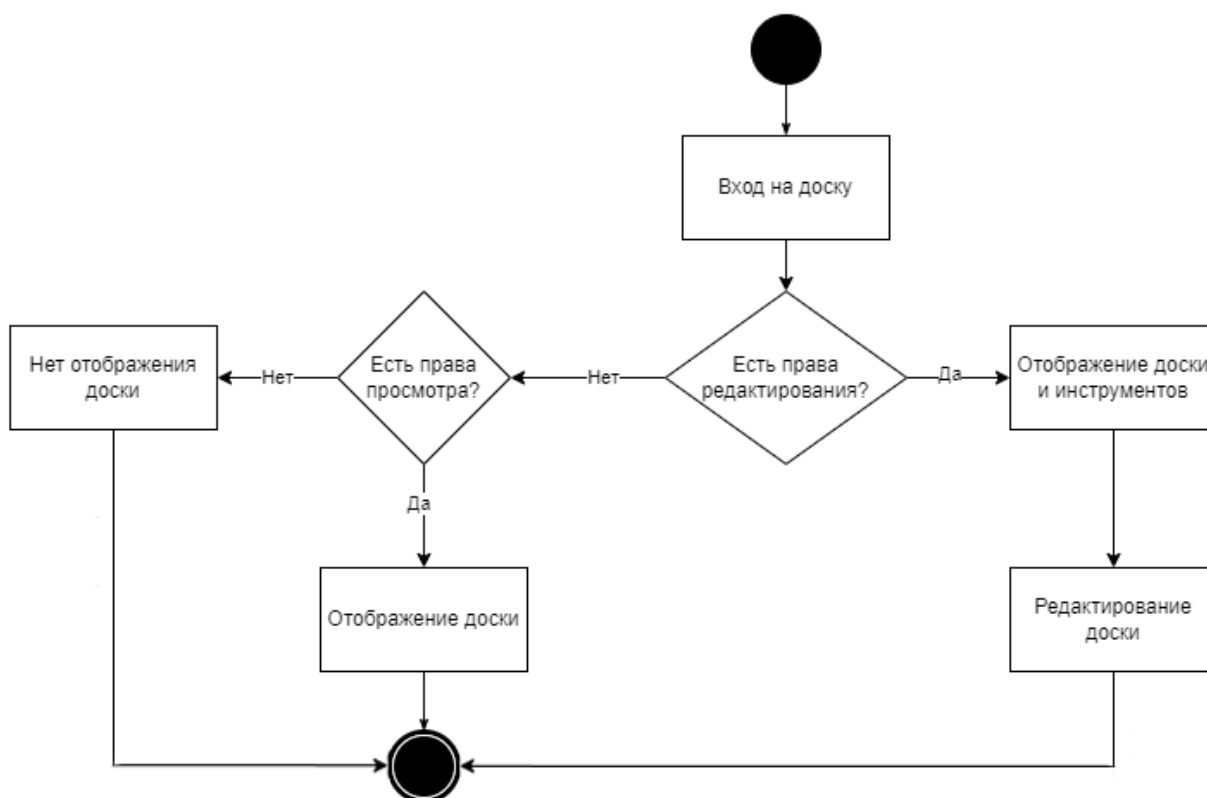


Рисунок 13. Диаграмма деятельности для алгоритма

Список и описание библиотек для реализации

1. Фронтенд (frontend)

- React — Библиотека для разработки пользовательских интерфейсов. Основная технология для работы с компонентами доски.
- Next.js — Фреймворк для React, предоставляющий поддержку серверного рендеринга и маршрутизации, а также улучшение производительности.
- Tailwind CSS — Утилитарный CSS-фреймворк для стилизации приложения.
- Zustand — Библиотека для управления состоянием приложения. Используется для хранения данных локального состояния.
- Liveblocks — Инструмент для работы с состоянием в реальном времени. Используется для синхронизации слоев доски между пользователями.
- Radix UI — Библиотека компонентов пользовательского интерфейса для создания доступных и настраиваемых элементов интерфейса.
- Axios — HTTP-клиент для взаимодействия с сервером. Используется для отправки запросов REST API.
- Lucide React — Библиотека иконок для использования в компонентах.
- Perfect Freehand — Утилита для создания плавных линий и кривых, используемых в режиме рисования.
- Prettier и ESLint — Инструменты для обеспечения стандартизации и качества кода.
- Jest и React Testing Library — Фреймворки для модульного тестирования компонентов и поведения приложения.
- Playwright — Библиотека для E2E-тестирования, проверяющая работу приложения с точки зрения пользователя.

2. Бэкенд (backend)

- Node.js — Серверная платформа для выполнения JavaScript-кода.
- Express.js — Веб-фреймворк для обработки маршрутов REST API и взаимодействия с фронтендом.
- Mongoose — ODM-библиотека для упрощенной работы с MongoDB.
- MongoDB — Документоориентированная база данных для хранения информации о досках, слоях и пользовательских данных.
- Multer — Библиотека для обработки загрузки файлов.
- Jest — Инструмент для модульного тестирования серверной логики.
- Supertest — Библиотека для тестирования API, позволяющая выполнять запросы к серверу и проверять ответы

Описание тестирования и результатов тестирования программной системы

Для написания unit-тестов использовался фреймворк Jest. Общее покрытие кода unit-тестами составило более 80%. Это свидетельствует о высоком уровне проверки модулей на предмет ошибок и соответствия требованиям.

Также нами было проведено E2E-тестирование, суть которого состоит в проверке всего программного обеспечения на предмет зависимостей, целостности данных, интерфейсами и базами данных для проверки успешного выполнения полного производственного сценария.

Для написания тестов был выбран фреймворк Playwright.

В ходе проведения тестирования были проверены следующие пути пользователя:

Авторизация пользователя:

- Проверка входа с корректными учетными данными.

Работа с организациями:

- Создание новой организации.
- Удаление организации.

Работа с досками:

- Создание новой доски.
- Редактирование существующей доски.
- Удаление доски.

Работа со элементами на доске:

- Добавление различных типов элементов (текст, изображение, фигуры).
- Редактирование содержимого элементов.
- Перемещение и изменение размеров элементов.
- Удаление элементов.

Совместная работа:

- Проверка реального времени обновления доски при одновременной работе нескольких пользователей.

Тесты запускались в среде Chrome и были успешно пройдены, что позволяет утверждать, что приложение работает корректно для конечного пользователя в соответствии с изначальной задумкой разработчиков.

Вывод:

В результате работы мы выполнили поставленную задачу - создали веб-приложение. Для достижения данной цели нами были реализованы следующие этапы: спроектированы основные страницы приложения, выбран стек технологий, на которых будет работать приложение, разработаны интерфейсы приложения, написана серверная часть. По завершении работы мы запустили приложение и проверили его работоспособность, а также провели тестирование, которое было успешно пройдено. Все вышеперечисленное позволило нам сделать вывод об успешном выполнении проекта.